

Assignment: Classification (SVM)

Task:

1.

- Load the dataset and convert categorical class labels under the target column to numerical values by using the LabelEncoder.
- Choose two features from dataset to apply SVM and Logistic Regression algorithms for classification. Plot the data by showing classes separately. Explain how and why you chose the two features?
- Classify testing data by using SVM and Logistic Regression classifiers. Provide accuracies.

2.

➤ **Shuffling and Splitting:**

- The dataset contains 569 samples.
- Shuffle all the dataset and split it into:
 - Training set: 400 samples
 - Validation set: 100 samples
 - Testing set: 69 samples

➤ **Preprocessing:**

- Standardize the numerical features (use StandardScaler from Scikit-learn).

➤ **Feature Selection:**

- Perform feature selection using correlation or other methods to identify the most important features.
- Visualize the dataset in a 2D plot with the chosen features, showing classes separately

3.

➤ **Linear Kernel and Decision Boundary:**

- Train an SVM classifier with a **linear kernel** and visualize the decision boundaries.
- Explain the results: Is the dataset linearly separable? How does SVM handle this?

➤ **RBF Kernel and Decision Boundary:**

- Train an SVM classifier with an **RBF kernel** and visualize the decision boundaries.
- Compare the results with the linear kernel. Discuss how the RBF kernel uses the **kernel trick** to map data into a higher-dimensional space.

➤ **poly Kernel and Decision Boundary:**

- Train an SVM classifier with a **poly kernel** and visualize the decision boundaries.
 - Compare the results with the linear kernel. Discuss how the poly kernel handle this?
- Compare kernels accuracy scores on the validation and testing sets using default hyperparameters.
- Use **grid search** to tune the following hyperparameters:
- **C**: Test values [0.01, 0.1, 1, 10, 100]. Plot the accuracy score on the validation set as **C** varies
 - **gamma**: Test values [0.001, 0.01, 0.1, 1]. Plot the accuracy score on the validation set as **gamma** varies.

4. Performance Analysis:

➤ **Training Time and Prediction Time:**

- Measure and analyze the training and prediction times for SVM under the following scenarios:
 - **Case 1**: 10% of the training set and kernel=linear, C=1.
 - **Case 2**: 100% of the training set and kernel=linear, C=1.
 - **Case 3**: 10% of the training set and kernel=rbf, C=1, gamma=0.01.
 - **Case 4**: 100% of the training set and kernel=rbf, C=1, gamma=0.01.
- **Visualization**: Plot a bar chart to show the training time and prediction time for each of these scenarios.

5. Exploring the Weaknesses of SVM

➤ **Overlapping Classes:**

- Manipulate the feature values (e.g., reduce separability between some wine quality scores) to create overlapping classes.
- Train the classifier again and discuss the impact on accuracy. How does the performance drop when classes overlap?

➤ **Noisy Data:**

- Add Gaussian noise to 10% of the samples in the training set.
- Retrain the SVM classifier with the RBF kernel and tuned parameters.
- Report accuracy on the validation and testing sets, and compare with the results from the clean dataset.

6. Conclusions:

After running your code, you can summarize:

- **SVM's strengths:** Good for high-dimensional spaces, especially when classes are separable.
- **SVM's weaknesses:** Struggles with noisy or overlapping data, leading to decreased accuracy.